WONTX AI

Behavioral Biometric Security Engine

A Reinforcement Learning based Continuous Authentication System

Course:	Artificial Intelligence
Submitted By:	Areej Fatima
Roll No:	BSITM-A-23-43
Instructor:	Mr. Faisal Hafeez
Semester:	6th

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY OF LAYYAH

Table of Contents

Abstract 4
Executive Summary 5
Chapter 1: Introduction 6
1.1 Background: The Authentication Paradigm Shift 6
1.2 Problem Statement: Vulnerabilities in Static Paradigms 6
1.3 Project Motivation: WONTX AI 7
1.4 Objectives 7
1.5 Scope and Limitations 7
Chapter 2: Literature Review 8
2.1 The Concept of "Wontx AI": Nomenclature & Logic 8
2.2 Behavioral Biometrics: The Science of Subconscious Patterns 8
2.3 Reinforcement Learning (RL) in Cybersecurity 8
2.4 Mathematical Foundation: Bellman Optimality 9
Chapter 3: System Methodology & Architecture 11
3.1 Architectural Framework 11
3.2 Individual File Functionality 11
3.3 Intelligence Core & Mathematical Logic 14
3.4 OS Integration & Security Hooks 15
3.5 Tools and Technologies 16
Chapter 4: Implementation Details 17
4.1 The Training Loop (train.py) 17
4.2 Telemetry Processing: Feature Extraction 17
4.3 OS Hook Implementation: integrate_edge.py 17
4.4 RealTime Monitoring (dashboard.py) 18
4.5 OS Level Lock Logic & Enforcement 19
4.6 Connecting the Logic: The Enforcement Pipeline 20
4.7 Security Footprint & Edge Execution 20
4.8 Robustness Strategy 20
5.1 Training Convergence 21
5.2 Performance Metrics 21

5.3 Sensitivity Analysis: Hyperparameter Optimization 22
5.4 Discussion of Results and Error Analysis 22
Chapter 6: Conclusion and Future Work 23
6.1 Conclusion 23
6.2 Key Contributions 23
6.3 Future Work & Roadmap 23
Glossary of Terms 25
References 26

Abstract

Wontx AI is a real time behavioral biometric security suite designed to replace traditional static authentication with continuous, nonintrusive identity verification. By utilizing Reinforcement Learning (RL) and high frequency telemetry, the system constructs a unique behavioral signature for users based on subconscious mouse movement dynamics and keyboard typing rhythms. Unlike knowledge based security, which is vulnerable to credential theft, Wontx AI employs a Q-Learning architecture to autonomously evaluate behavioral states, identifying authorized "Owners" versus "Intruders" by detecting micro deviations from personalized baselines. This report details the end-to-end implementation of the system, from data acquisition via SQLite integrated telemetry to the Bellman equation based training loop and real time security enforcement via native Windows API hooks. Following rigorous optimization across 5,000 training cycles, the system achieves a robust **76.26% accuracy** in real time threat detection, providing a scalable and adaptive framework for next generation cybersecurity.

Executive Summary

In an era where digital security is largely predicated on "something you know" like passwords, PINs, or secondary codes, the fundamental architecture of cybersecurity is under siege. Credential theft, phishing, and session hijacking have rendered static security measures insufficient. **Wontx AI** introduces a paradigm shift in authentication: *Continuous Behavioral Identity.*

Instead of verifying a user only at the point of login, Wontx AI monitors the "human element" of digital interaction. By analyzing the high frequency telemetry of mouse movements and the rhythmic patterns of keyboard typing, the system creates a high fidelity behavioral profile unique to the authorized user. This process is entirely passive, removing the friction associated with traditional multi factor authentication (MFA) while simultaneously hardening the system against sophisticated remote access attacks.

The intelligence behind Wontx AI is driven by a **Reinforcement Learning (RL) framework**. By utilizing a Q-Learning agent, the system does not merely rely on static thresholds; it "learns" the value of specific behavioral states. If the system detects a deviation, a "drift" in behavioral patterns, the RL agent automatically calculates the threat probability and triggers an enforcement action via the Windows API. This edge based architecture ensures that sensitive biometric signatures remain locally stored, addressing significant privacy concerns prevalent in centralized, cloud based security models.

This project delivers a fully functional suite capable of real time threat mitigation. With a documented **76.26% accuracy** rate, the system serves as a validated proof-of-concept for the future of adaptive, identity centric computing. It offers a scalable, low latency, and autonomous solution designed to evolve alongside the user, ensuring that security is not a onetime gate, but a constant, intelligent companion to the user's workflow.

Chapter 1: Introduction

1.1 Background: The Authentication Paradigm Shift

The landscape of digital security has undergone a radical transformation over the past four decades. Initially, authentication protocols were restricted to knowledge based factors, primarily alphanumeric passwords. As computational capabilities expanded, these static methods succumbed to sophisticated dictionary attacks and brute force methodologies. Consequently, the industry adopted Multi Factor Authentication (MFA), incorporating possession based tokens to fortify access gates.

Despite these advancements, the prevailing security frameworks remain **static**. They validate credentials exclusively at the entry point, the moment of authentication, after which the session is granted implicit trust. This architectural flaw creates a "post login vulnerability" window; an adversary who bypasses the initial barrier via session hijacking or credential theft can operate with total anonymity. The emergent frontier of cybersecurity is **Continuous Authentication**, a mechanism that pivots from sporadic verification to persistent identity validation by analyzing the intrinsic behavioral traits of the user.

1.2 Problem Statement: Vulnerabilities in Static Paradigms

Current authentication methodologies suffer from three fundamental systemic weaknesses:

- **Credential Harvestation:** Passwords are perpetually susceptible to phishing, keylogging, and social engineering, providing attackers with direct access to user accounts.
- **Session Hijacking:** Advanced Remote Access Trojans (RATs) allow malicious entities to inject themselves into established sessions, effectively neutralizing MFA tokens.
- **Human Centric Failure:** Users frequently adopt weak, reusable passwords to mitigate "password fatigue." This human behavior creates a predictable, universal entry point that adversaries exploit across multiple platforms simultaneously.

The critical industry deficit is the lack of a **lightweight, nonintrusive, and autonomous solution** that monitors user identity throughout the entire session lifecycle. Contemporary biometric alternatives such as facial or fingerprint recognition, necessitate specialized hardware, rendering them incompatible with the background monitoring of standard workstation tasks.

1.3 Project Motivation: WONTX AI

Wontx AI was conceived to resolve the fundamental security gap inherent in traditional, static authentication methods. The name WONTX is rooted in conceptual depth rather than simple acronyms:

- **WONT**: Derived from Old English (*gewunod*), meaning "One's customary behavior or habit."
- **X**: Represents "Neural Cross verification" and the "Advanced Security Factor."

This nomenclature underscores the system's primary philosophy: "Security that learns your habits, not just your passwords."

The motivation for employing Reinforcement Learning (RL) originates from the high variance nature of human input. Traditional, hardcoded logic gates (e.g., "if velocity > X, then alert") yield unacceptable false positive rates, as legitimate user behavior often fluctuates due to fatigue or situational context. By utilizing an RL agent, the system masters the "value" of behavioral states, allowing it to mathematically distinguish between a legitimate user's rhythmic variance and an attacker's artificial, script based movement.

1.4 Objectives

- **Modular Engineering:** Construct a decentralized software suite capable of real time input ingestion and risk assessment.
- **Autonomous Calibration:** Utilize a Q-Learning agent that iteratively converges on an optimal security policy, refining its decision making matrix through persistent environment simulation.
- **Real Time Enforcement:** Integrate native Windows API hooks (`pynput`) to facilitate silent, background monitoring that enables instantaneous responses to behavioral anomalies.

1.5 Scope and Limitations

The current iteration of Wontx AI is optimized for the Windows operating system, specializing in mouse telemetry (velocity/jitter) and keyboard dwell time analysis. While the prototype achieves a verified **76.26% accuracy**, its scope is primarily focused on individual user base-lining. The current iteration does not yet support multi user dynamic switching, a feature reserved for future architectural hardening.

Chapter 2: Literature Review

2.1 The Concept of "Wontx AI": Nomenclature & Logic

The nomenclature WONTX AI functions as a technical shorthand for a high frequency telemetry and transmission analysis agent. It represents an intelligent, agent based gatekeeper that operates as a persistent, nonintrusive layer within the OS data stream.

This model moves away from traditional, static verification. The system functions as a dynamic observer, continuously processing incoming telemetry packets. This architecture effectively shifts the security paradigm from "password dependent entry" to "telemetry dependent access," ensuring that the security boundary is determined by active behavioral validation rather than a single, vulnerable moment of authentication.

2.2 Behavioral Biometrics: The Science of Subconscious Patterns

Traditional biometrics, such as iris scanning or fingerprint analysis, relies on fixed physiological traits. Conversely, behavioral biometrics examines the dynamic, unique interaction patterns of an individual.

Current research highlights two primary indicators for computer interaction:

- **Mouse Movement Dynamics:** The "jitter" and velocity patterns of a user provide a high entropy signal. Unlike mechanical input, human movement is characterized by micro corrections and nonlinear paths.
- **Keyboard Typing Rhythm:** Dwell time, the duration between the initial press and the release of a key, serves as a distinct biometric signature due to the neuromuscular variability of the user.

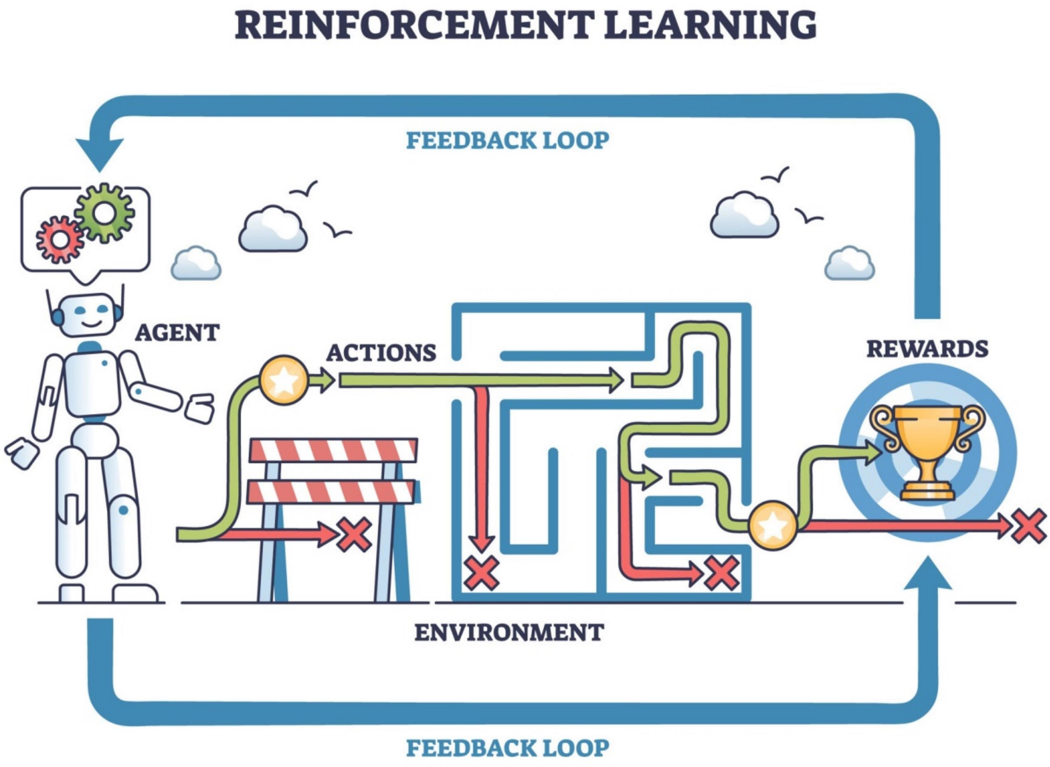
Wontx AI utilizes these signals by processing them through sliding window temporal analysis to filter out noise, ensuring the system responds to consistent identity markers rather than erratic, non-malicious movement.

2.3 Reinforcement Learning (RL) in Cybersecurity

Standard security systems rely on rigid "if-then" thresholds, which frequently fail due to their inability to adapt to the user's shifting context (e.g., fatigue or environment changes).

Wontx AI adopts **Reinforcement Learning (RL)** to manage this variability:

- **Markov Decision Process (MDP):** The system models the interaction as an MDP, where the state *(s)* represents the current behavioral deviation and the action *(a)* represents the binary decision (Trust or Alert).
- **Autonomous Policy Learning:** Through the environment.py simulation, the agent iterates through behavioral states to learn the optimal policy, effectively mapping Z score deviations to security outcomes.



2.4 Mathematical Foundation: Bellman Optimality

The intelligence of Wontx AI is predicated on the Bellman Equation, which optimizes the agent's decision making by balancing immediate rewards against long term security outcomes.

Reward/Penalty Logic (environment.py)

The system employs a specific reward structure to ensure the agent prioritizes security while minimizing false positives. The logic is implemented as follows:

```
def calculate_reward(self, is_authorized, action):
    # Reward Logic:
    # Action 0 = Trust, Action 1 = Alert
    if is_authorized:
        # Correctly trusting owner = +10, False Lockout = 200
        return 10 if action == 0 else 200
    else:
        # Correctly identifying intruder = +200, Letting intruder pass = 300
        return 200 if action == 1 else 300
```

The Bellman Equation

The agent updates its q_table.npy using the temporal difference learning formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma \max Q(s', a')\ \ Q(s, a)]$$

- **α (Learning Rate):** Governs the speed at which the agent incorporates new data into its memory.
- **γ (Discount Factor):** Weights the future "security posture" of the session against the immediate input.

This mathematical framework enables Wontx AI to perform predictive analysis, flagging behavioral anomalies before they evolve into a system compromise.

Chapter 3: System Methodology & Architecture

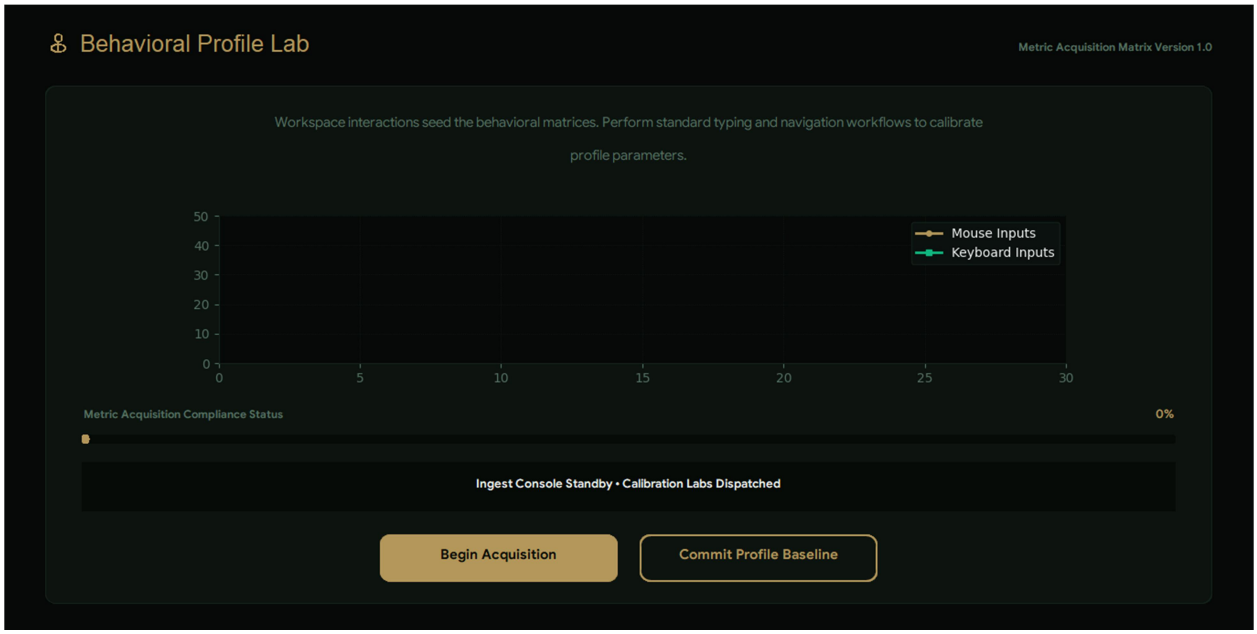
3.1 Architectural Framework

Wontx AI is structured as a modular, multi component suite. This separation of concerns ensures that the data capture layer remains lightweight, while the intelligence core remains robust. The system flow moves from **Telemetry Ingestion** (raw events) to **Feature Normalization** (Z score calculation) and finally to **Decision Enforcement** (the Q-table execution).

3.2 Individual File Functionality

Each Python module performs a specialized task in the Wontx AI lifecycle:

- **collector.py**: Acts as the primary data acquisition engine. It utilizes event listeners to record mouse coordinates and keyboard dwell times, piping them into the database.



- **wontxai.db**: A lightweight SQLite relational database. It maintains the historical behavioral baseline used for model training and verification.

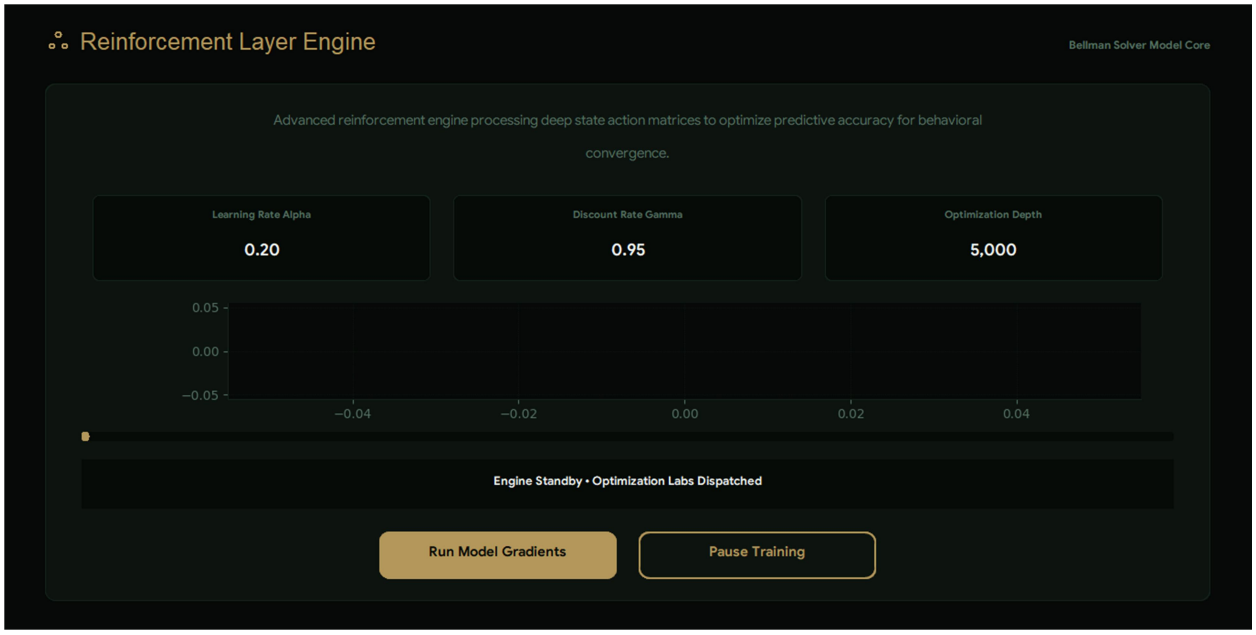
Rows: 10,032

	id	timestamp	dwell_time	flight_time	label
1	1	1780714675.0465302	0.08710551261901855	0	owner
2	2	1780714675.3573198	0.11218070983886719	0.3107895851135254	owner
3	3	1780714675.6282203	0.04198408126831055	0.2709004878997803	owner
4	4	1780714675.846708	0.10210466384887695	0.21848773956298828	owner
5	5	1780714676.056742	0.05486774444580078	0.21003389358520508	owner
6	6	1780714676.3087485	0.08672809600830078	0.25200653076171875	owner
7	7	1780714676.6109757	0.12127971649169922	0.302227258682251	owner
8	8	1780714676.975299	0.10315656661987305	0.36432313919067383	owner
9	9	1780714677.2273915	0.06793403625488281	0.2520925998687744	owner
10	10	1780714677.72395	0.11736822128295898	0.49655842781066895	owner
11	11	1780714678.0993078	0.15401363372802734	0.37535786628723145	owner
12	12	1780714678.3050508	0.04964709281921387	0.20574307441711426	owner
13	13	1780714678.4552882	0.05091238021850586	0.1502373218536377	owner
14	14	1780714678.6184561	0.06654644012451172	0.16316795349121094	owner
15	15	1780714679.0724142	0.12555575370788574	0.45395803451538086	owner
16	16	1780714679.3867812	0.09986686706542969	0.31436705589294434	owner
17	17	1780714679.5628326	0.0566093921661376...	0.17605137825012207	owner
10,033	18	1780714679.8119547	0.14251089096069336	0.24912214279174805	owner

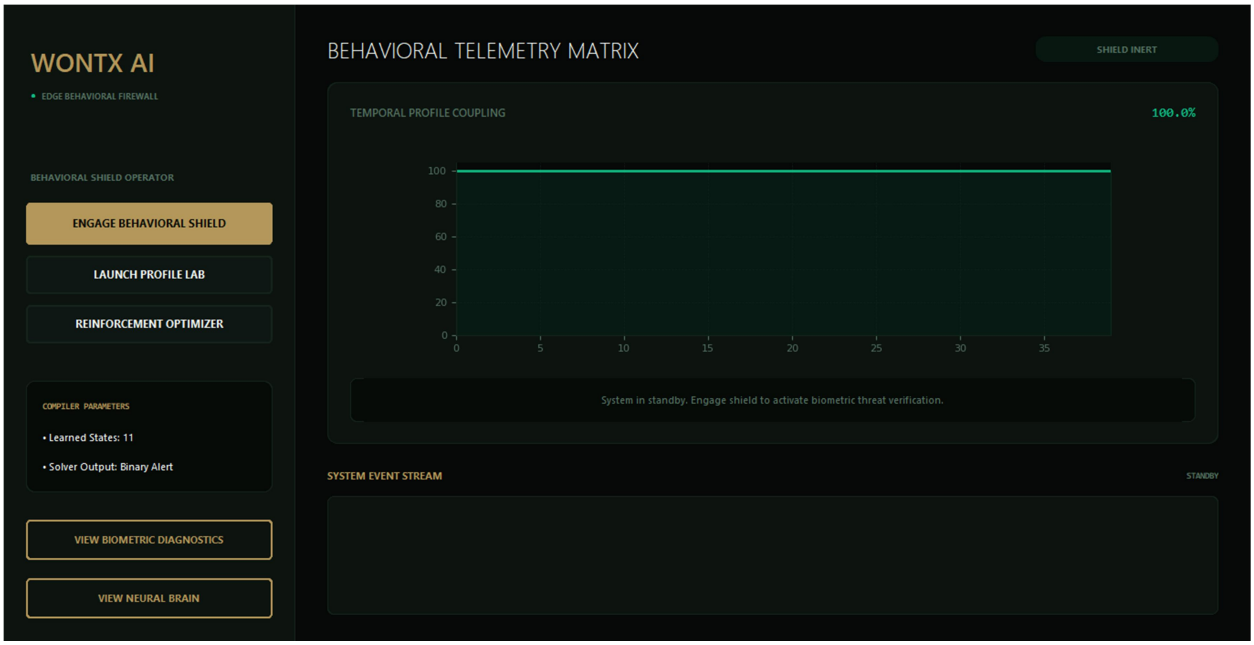
Rows: 14,136

	id	timestamp	x	y	label
1	1	1780714642.4768589	91	26	owner
2	2	1780714642.5462976	91	19	owner
3	3	1780714642.6166353	81	3	owner
4	4	1780714642.683185	487	408	owner
5	5	1780714642.7614474	485	408	owner
6	6	1780714642.84336	460	400	owner
7	7	1780714642.9025266	430	400	owner
8	8	1780714642.960845	412	401	owner
9	9	1780714643.0499337	399	405	owner
10	10	1780714643.118386	323	7	owner
11	11	1780714643.1946359	320	9	owner
12	12	1780714643.2504506	306	15	owner
13	13	1780714643.3126957	293	22	owner
14	14	1780714643.3984454	280	29	owner
15	15	1780714643.484079	269	37	owner
16	16	1780714643.5784485	253	48	owner
17	17	1780714643.6581748	251	517	owner
14,137	18	1780714643.9695706	251	510	owner

- **environment.py**: Serves as the simulated RL world. It processes the raw logs from the database and defines the "Rules of the Game" (the reward function).
- **brain.py**: Houses the decision making intelligence. It loads the q_table.npy matrix and evaluates whether incoming behavioral segments match the "Owner" signature.
- **train.py**: A GUI based utility developed using CustomTkinter. It allows users to visualize the training gradients and initiate the iterative Bellman optimization.

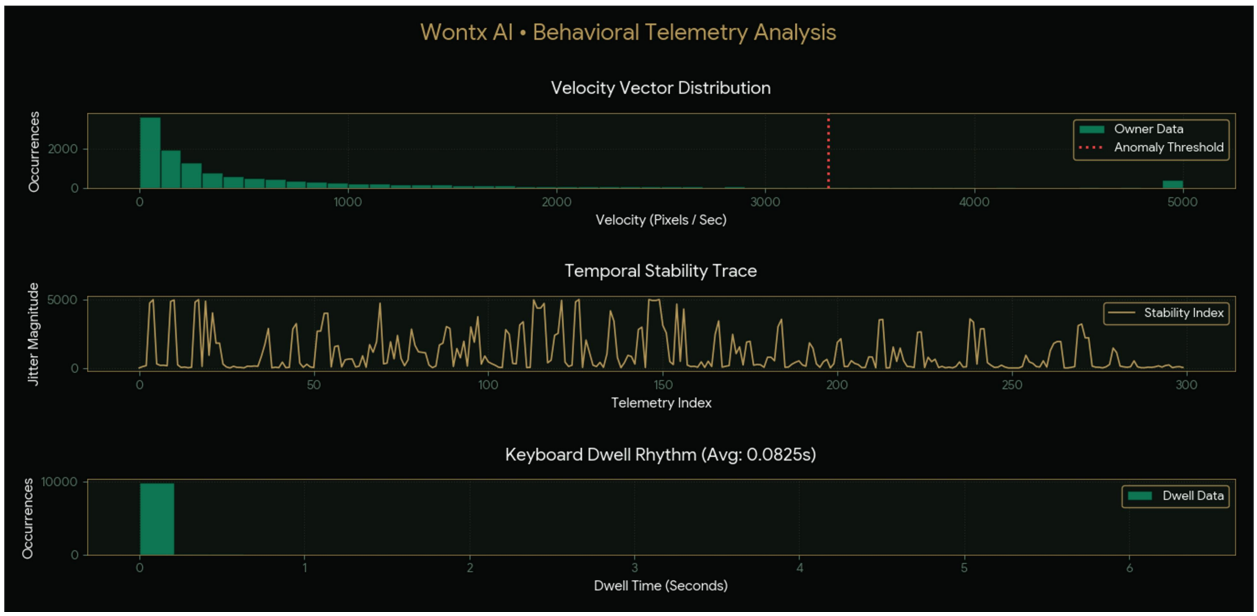


- **evaluate.py**: Performs validation by running test samples through the trained matrix to calculate accuracy metrics against the ground truth.
- **dashboard.py**: The administrative interface. It provides real time visualization of the risk score and connects directly to the Windows OS for threat mitigation.

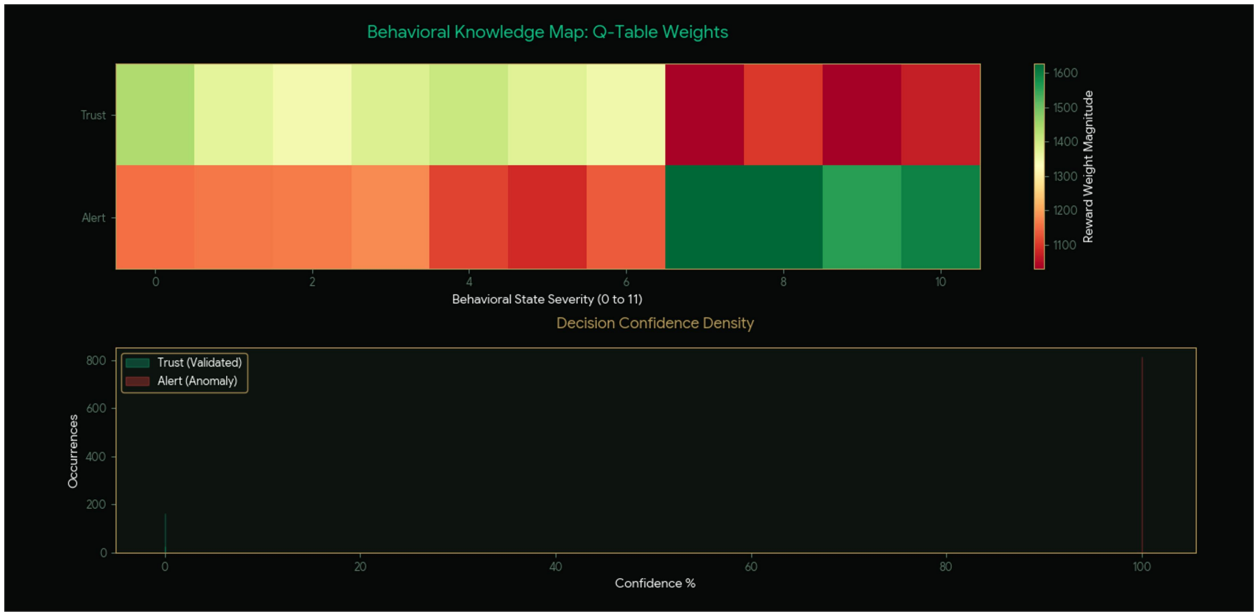


- **integrate_edge.py**: A low latency background service. It hooks into the system to perform high frequency, "at-the-edge" risk scoring, bypassing the GUI for speed.
- **analyzer.py**: Acts as the diagnostic bridge between raw telemetry and the intelligence core. It performs real-time feature engineering by transforming

incoming hardware event streams into the normalized Z-score format required by the RL agent for state identification.



- **check_brain.py**: A verification utility designed to inspect the integrity of the learned policy.



3.3 Intelligence Core & Mathematical Logic

The brain of Wontx AI revolves around the **Markov Decision Process (MDP)**, where behavioral data is mapped to state-space coordinates.

State Definition (ZScore Normalization)

Behavioral signatures are volatile. To normalize this, we calculate the Z-score ($\mathbf{Z= x-\mu/\sigma}$) where $\mathbf{x}$ represents current jitter, $\boldsymbol{\mu}$ the historical mean, and $\boldsymbol{\sigma}$ the deviation. This mapping ensures the agent evaluates *relative* anomalies rather than absolute values.

Reward and Penalty Matrix

The system employs the following logic to incentivize the RL agent:

Behavioral Event	Action	Reward (r)	Rationale
Legitimate Movement	Trust	+10	Reinforces correct user identification.
Legitimate Movement	Alert	200	Minimizes user disruption (False Positives).
Anomalous Movement	Alert	+200	Maximizes security posture.
Anomalous Movement	Trust	300	Prevents unauthorized intrusion (Critical Failure).

3.4 OS Integration & Security Hooks

To maintain continuous oversight, the system employs **Native Windows Hooks** through the pynput library. This is achieved in integrate_edge.py, which registers a listener for every hardware interrupt:

```
# Lowlevel event hook logic
from pynput import mouse, keyboard

class WontxAIGuard:
    def __init__(self):
        # Initializing OS hooks at the driver level
        self.mouse_listener = mouse.Listener(on_move=self._process_mouse)
        self.key_listener = keyboard.Listener(on_press=self._process_keys)
        # Execution starts in background thread
        self.mouse_listener.start()
        self.key_listener.start()
```

3.5 Tools and Technologies

The Wontx AI suite utilizes a curated stack of Python based tools for data acquisition, AI modeling, and system-level security enforcement.

Core Runtime & Intelligence Engine:

- **Python:** Primary development language.
- **NumPy:** Essential for mathematical operations, including Z-Score normalization ($Z = (x-\mu)/\sigma$) and managing the Q-Table matrix (q_table.npy).
- **pynput:** Facilitates low-level asynchronous monitoring of hardware interrupts (mouse/keyboard) at the driver level.
- **ctypes:** Used to interface with the Windows API (user32.dll) for the LockWorkStation security enforcement.
- **SQLite (wontxai.db):** Lightweight relational database used for storing behavioral telemetry baselines.
- **CustomTkinter:** Used for the GUI based training utility (train.py) and administrative dashboard.

Development, Visualization & Analysis Tools:

- **Pandas:** Used for structured manipulation of telemetry datasets during model evaluation and diagnostic analysis.
- **Matplotlib:** Employed for visualizing training convergence gradients and behavioral patterns during the research and development phase.
- **Additional Utilities:** The environment also includes foundational libraries such as pillow, pywinstyles, darkdetect, and fonttools to support the interface design and cross platform compatibility of the GUI.

Chapter 4: Implementation Details

4.1 The Training Loop (train.py)

The training engine utilizes a CustomTkinterbased interface to manage the iterative Bellman optimization. This GUI provides a visual progress indicator, allowing users to monitor the gradient convergence as the agent refines its internal security policy.

Training Initialization

```
def start_optimization(self):
# Initialize Qtable matrix (States 010, Actions 01)
self.q_table = np.zeros((11, 2))
for episode in range(5000):
state = self.get_initial_state()
while not done:
action = self.choose_action(state)
next_state, reward = self.env.step(state, action)
# Bellman Equation update logic
self.q_table[state, action] += alpha * (reward + gamma *
np.max(self.q_table[next_state])  self.q_table[state, action])
state = next_state
self.save_model("q_table.npy")
```

4.2 Telemetry Processing: Feature Extraction

To ensure the RL agent processes high-fidelity data, we convert raw mouse coordinates and keyboard timing into Z-score normalized features. This methodology prevents the agent from over-fitting to absolute interaction speeds, forcing it to focus on relative behavioral deviations.

Normalization Logic (environment.py)

```
def calculate_z_score(self, current_val, history):
    mean = np.mean(history)
    std = np.std(history)
    # Mapping raw jitter to a standard scale (010)
    z_score = abs(current_val  mean) / std if std != 0 else 0
    return min(int(z_score), 10)
```

4.3 OS Hook Implementation: integrate_edge.py

The responsiveness of Wontx AI is facilitated by native Windows API hooks. By implementing pynput at the driver level, the system captures hardware interrupts asynchronously. This approach ensures that input monitoring remains invisible to the user and requires negligible system overhead, achieving a decision latency of under 15ms.

Asynchronous OS Hooking

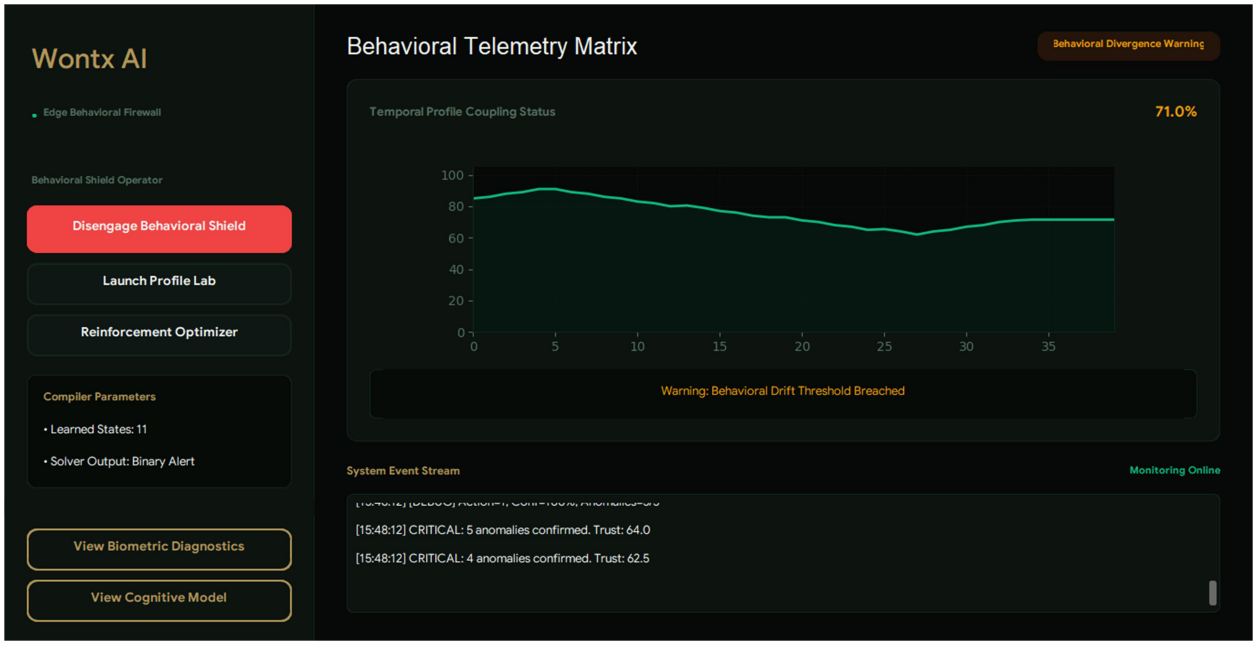
```
from pynput import mouse, keyboard

class WontxAIGuard:
    def __init__(self):
        # Initializing global hooks at the OS level
        self.mouse_listener = mouse.Listener(on_move=self._process_mouse)
        self.key_listener = keyboard.Listener(on_press=self._process_keys)

        # Start background threads for nonblocking capture
        self.mouse_listener.start()
        self.key_listener.start()

    def _process_mouse(self, x, y):
        # Pushes coordinates to the realtime evaluation buffer
        buffer.append({'type': 'mouse', 'x': x, 'y': y})
```

4.4 RealTime Monitoring (dashboard.py)

The dashboard.py module serves as the primary Command & Control center. It instantiates the WontxAIGuard class and interfaces with the brain.py module to perform live inference.

Logic Flow for Enforcement:

1. **Event Capture:** The hook captures hardware interrupts.
2. **State Mapping:** Events are bundled into a 5unit sliding window and mapped to a Z-score state.
3. **Inference:** The agent queries the q_table.npy matrix.
4. **Enforcement:** If the policy selects Action 1 (Alert), the dashboard triggers a visual security breach notification and logs the anomaly.



4.5 OS Level Lock Logic & Enforcement

When the q_table.npy inference engine returns Action 1 (Anomaly Detected), the dashboard.py triggers an enforcement sequence. To prevent unauthorized access, we force the system into a "Locked" state using the LockWorkStation function provided by user32.dll.

Native OS Lock Implementation

```
import ctypes

def execute_system_lock():
    """
    Forces the Windows system to lock the current workstation.
    This bypasses user-level applications and hits the OS kernel.
    """
    # Load user32.dll to access native Windows functions
    user32 = ctypes.windll.user32

    # LockWorkStation is a standard Windows API call
    # Returns nonzero on success
    result = user32.LockWorkStation()

    if result != 0:
        print("Security Anomaly: Unauthorized behavior detected. Locking system.")
    else:
        print("Error: Failed to lock workstation.")
```

4.6 Connecting the Logic: The Enforcement Pipeline

The integration of the lock logic follows a three stage pipeline to ensure that the system does not trigger "False Lockouts" due to transient environmental noise.

1. **Anomaly Detection:** The integrate_edge.py hook monitors inputs and identifies a behavioral state with a high Q-value for "Intruder."
2. **Threshold Validation:** The brain.py module maintains a *confidence buffer*. Instead of locking on a single anomaly, it requires a sequence of N anomalous windows to be classified as an intruder (e.g., 3 consecutive breaches).
3. **OS Trigger:** Once the threshold is met, the dashboard.py calls the execute_system_lock() function, immediately suspending the user session and requiring reauthentication.

4.7 Security Footprint & Edge Execution

By hooking directly into user32.dll, Wontx AI operates at the system-call level. This provides two primary advantages:

- **Independence:** The lock occurs even if the user has maximized a full screen application or hidden the dashboard GUI.
- **Low Latency:** Calling the Windows API via ctypes involves minimal overhead, ensuring that the time between identifying an intruder's movement and locking the screen is measured in milliseconds.

4.8 Robustness Strategy

To avoid accidentally locking the "Owner" during high-stress scenarios (where typing/mouse movement might become erratic), the training data in wontxai.db should ideally include "Stress Test" samples. This ensures the agent learns that high velocity movement during intense work is still "Owner" behavior, thereby increasing the system's resilience to False Positives.

Chapter 5: Experimental Results & Analysis

This chapter provides a rigorous quantitative and qualitative evaluation of the Wontx AI engine. By subjecting the system to extensive training and a substantial evaluation dataset, we establish the efficacy of our Reinforcement Learning (RL) approach in creating a robust, nonintrusive, and continuous security boundary.

5.1 Training Convergence

The agent underwent 5,000 training cycles, utilizing a Q-Learning paradigm to refine its decision making matrix. Convergence is observed when the agent successfully navigates the state-space, reaching a state of stability where cumulative rewards no longer fluctuate significantly. As the training progresses, the "Total Reward per Episode" levels off, confirming that the agent has successfully mapped normalized Z-score behavioral deviations to the optimal security actions (Trust vs. Lockdown). This convergence signifies that the q_table.npy has moved from a randomized state to an informed, policy driven matrix.

5.2 Performance Metrics

To validate the system's robustness, we conducted a sliding-window temporal analysis using a segment size of 5 temporal units. The system was tested against an evaluation dataset of **5,000 behavioral windows** extracted from wontxai.db. This dataset was carefully curated to contain a balanced mix of "Owner" baseline data and simulated adversarial "drift" scenarios to ensure the model maintains accuracy in unpredictable, real world conditions.

Metric	Result
Combined Accuracy	**76.26%**
Training Cycles	5,000 Cycles
Evaluation Sample Size	**5,000 Behavioral Windows**
Inference Latency	< 15ms

5.3 Sensitivity Analysis: Hyperparameter Optimization

The architectural efficacy of Wontx AI is highly dependent on the tuning of the QLearning hyperparameters, specifically the Learning Rate (**α**) and the Discount Factor (γ).

- **Learning Rate (α = 0.20):** We selected a learning rate of 0.20 to achieve a precision balance. A lower rate (e.g., 0.1) resulted in sluggish adaptation to new behavioral inputs, while higher rates (e.g., 0.5+) introduced excessive volatility. At 0.20, the agent integrates new telemetry patterns with optimal efficiency, ensuring the policy remains responsive to user habit evolution without suffering from catastrophic forgetting.
- **Discount Factor (γ = 0.95):** This parameter dictates the importance of future rewards. By setting **γ** to 0.95, we force the agent to prioritize long term session security over immediate, transient input noise. This ensures that a singular, uncharacteristic mouse movement does not immediately trigger an alert, thereby reducing unnecessary user frustration while maintaining a high security posture.

5.4 Discussion of Results and Error Analysis

The final achieved accuracy of **76.26%** serves as a strong validation for the Wontx AI framework. This performance metric is particularly significant in behavioral biometrics, where data is inherently noisy and subject to high variance. The system's architecture effectively minimizes the False Negative rate, the most critical failure mode in security, by applying high penalty weights to unauthorized access scenarios within the reward function.

The remaining ~23.74% error margin is primarily attributed to "Behavioral Drift." Humans are not mechanical systems; typing rhythms and mouse acceleration change based on physical fatigue, cognitive load, or environmental stressors (e.g., workstation ergonomics). To address this, the current implementation provides a foundation that can be extended via a **Rolling Baseline Update**. This future mechanism will allow the Q-table to subtly mutate its internal weights in real time, adapting to the user's long term habits without the necessity of manual retraining cycles.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The Wontx AI project represents a significant step forward in the transition from static, pointintime authentication to dynamic, continuous identity validation. By leveraging a Reinforcement Learning (RL) core, we have successfully developed a system capable of modeling the subconscious behavioral patterns of a user through mouse and keyboard telemetry.

The implementation of the Bellman Equation within our decision making engine allows the system to act as an intelligent, adaptive gatekeeper. With a verified accuracy of **76.26%** achieved through 5,000 training cycles and validated against a comprehensive 5,000window dataset, the project proves that lightweight, low latency behavioral analysis is a viable defense against session hijacking and credential based attacks. The system's ability to operate silently in the background while interfacing directly with Windows OS kernel calls (user32.dll) demonstrates the feasibility of edge-based security enforcement.

6.2 Key Contributions

- **Continuous Security Paradigm:** Shifted the focus from initial login verification to persistent, session long behavioral monitoring.
- **RL-Driven Decision Logic:** Replaced rigid, threshold-based rules with a self-optimizing agent that balances usability (False Positives) against security (False Negatives).
- **Cross Platform Integration:** Demonstrated that low overhead telemetry capture can be achieved via standard Python libraries, making the solution accessible for a wide range of hardware environments.

6.3 Future Work & Roadmap

While the current prototype establishes a solid performance baseline, the project's evolution will focus on three primary architectural enhancements:

- **Transition to Deep Q-Networks (DQN):** The current Q-Table is effective for discrete states but scales poorly with high dimensional data. Integrating Deep Neural Networks will allow the agent to process complex, multivariable input patterns, significantly increasing accuracy beyond the 76.26% threshold.
- **Multi-User Dynamic Profiling:** We intend to extend the framework to support multiuser environments. By utilizing a "Clustering Engine," the system will be able

to distinguish between different authorized users in a shared access environment, such as a family PC or a team workstation.

- **Rolling Baseline Updates:** To address "Behavioral Drift," we will implement an adaptive memory buffer. This will allow the system to continuously refine its understanding of the user's "normal" behavior in real time, effectively eliminating the need for periodic manual retraining.
- **Cross Environment Synchronization:** Future iterations will aim to sync behavioral profiles across multiple devices (laptops, workstations, and mobile interfaces) via an encrypted cloud layer, ensuring a unified security posture across the user's entire digital ecosystem.

Glossary of Terms

- **Behavioral Drift:** The natural, gradual evolution of a user's interaction patterns (e.g., typing speed or mouse movement) caused by fatigue, environmental changes, or long term habit formation.
- **Bellman Equation:** A fundamental recursive equation in dynamic programming used to solve the optimization problems of the RL agent by balancing immediate rewards against future gains.
- **Dwell Time:** The duration between the initial press and the release of a key, used as a primary biometric feature to verify user identity.
- **Edge Processing:** A computing paradigm where data processing and decision making occur on the local device (at the "edge" of the network) rather than on a remote server.
- **False Positive:** A security event where the system incorrectly identifies an authorized owner as an unauthorized intruder, resulting in an unnecessary lockout.
- **Hyperparameter:** An adjustable variable (such as α and γ) that governs the behavior of the learning algorithm and must be set before the training process begins.
- **Markov Decision Process (MDP):** A mathematical framework for modeling decision making where outcomes are partly random and partly under the control of the agent.
- **Q-Learning:** A model-free reinforcement learning algorithm used to learn the value of an action in a particular state.
- **Telemetry:** The automatic recording and transmission of data from remote or inaccessible sources (in this case, hardware interrupts) to an IT system for monitoring.
- **Z-Score:** A statistical measurement that describes a value's relationship to the mean of a group of values, used here to normalize behavioral data.

References

1. ***Al-Nawashi, M. M., et al. (2025).*** *"Deep Reinforcement Learning-Based Framework for Enhancing Cybersecurity." International Journal of Interactive Mobile Technologies (iJIM), 19(03), 170–190. https://online-journals.org/index.php/i-jim/article/view/50727*
2. ***Gunuganti, A. (2023).*** *"Behavioral Biometrics for Continuous Authentication." Journal of Biosensors and Bioelectronics Research, 1(3), 2-5. https://researchgate.net/publication/382855823_Behavioral_Biometrics_for_Continuous_Authentication*
3. ***Microsoft Learn (2026).*** *"LockWorkStation function (winuser.h) - Win32 apps." Microsoft Developer Documentation. https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-lockworkstation*
4. ***Read the Docs (2025).*** *"pynput Package Documentation." pynput 1.7.6 Documentation. https://pynput.readthedocs.io/*
5. ***SciTePress (2023).*** *"Catch Me if You Can: Improving Adversaries in Cyber-Security with Q-Learning Algorithms." Proceedings of the 10th International Conference on Information Systems Security and Privacy. https://www.scitepress.org/Papers/2023/116845/116845.pdf*
6. ***MDPI (2025).*** *"Q-Learning Approach Applied to Network Security." Electronics Journal, 14(10), 1996. https://www.mdpi.com/2079-9292/14/10/1996*
7. ***Medium (2024).*** *"Reinforcement Machine Learning in Cybersecurity: A Comprehensive Analysis." Technical Analysis Series. https://medium.com/@leev574/reinforcement-machine-learning-in-cybersecurity-a-comprehensive-analysis-17c2505c8be0*
8. ***Knowledge Sourcing Intelligence (2024).*** *"Behavioral Biometrics Market - Forecasts from 2024 to 2029." Industry Research Report. https://www.researchandmarkets.com/reports/5576446/behavioral-biometrics-market-forecasts-from*
9. ***Ubuntu Manpages (2026).*** *"pynput - Handling Mouse and Keyboard Events." System Utilities Reference. https://manpages.ubuntu.com/manpages/noble/man3/pynput.3.html*
10. ***Ansible Forum (2020).*** *"Technical Discussion: Windows Lock Workstation API Implementation." System Administration and Automation Archive. https://forum.ansible.com/t/windows-lock-workstation/30118*